AWS VPC PEERING

Student Project Report

Prepared by: **Milind Kailas Tajane**

Environment: Amazon Web Services (AWS)
Region: Asia Pacific (Mumbai — ap-south-1)

Project summary

This project demonstrates private network communication between two Amazon Virtual Private Clouds (VPCs) using VPC Peering. Route tables direct traffic across the peering connection, and security group rules control which traffic is allowed. EC2 instances were used to test connectivity in both directions and to demonstrate the effect of removing and restoring an ICMP rule.

1. Objectives

The objectives of this project were to configure two VPCs, connect them using VPC Peering, route traffic between the VPC CIDR ranges, apply security group controls, and validate network connectivity using ICMP ping tests.

2. Network design

The lab uses two IPv4 VPC networks in the AWS Mumbai region:

Component	Configuration
VPC A	ewb-vpc-a — 10.10.0.0/16
VPC B	ewb-vpc-b — 10.20.0.0/16
Public subnet (VPC A)	ewb-a-public — 10.10.1.0/24
Private subnet (VPC A)	ewb-a-private — 10.10.2.0/24
Private subnet (VPC B)	ewb-b-private — 10.20.1.0/24
VPC Peering	ewb-a-b-peer — Active
Internet Gateway	ewb-igw-a — attached to VPC A

3. Routing and security

The public route table for VPC A contains a default route to the Internet Gateway and a route for 10.20.0.0/16 through the peering connection. The private route table in VPC A routes 10.20.0.0/16 through the peering connection. The private route table in VPC B routes 10.10.0.0/16 through the peering connection. Security groups permit SSH only from the intended bastion source and ICMP between the test hosts as configured for the lab.

4. EC2 test hosts

The lab used a bastion host in VPC A's public subnet, App-A in VPC A's private subnet, and App-B in VPC B's private subnet. SSH agent forwarding allowed access to the private instances without copying the private key onto the bastion host.

5. Validation and results

ICMP tests showed successful communication from App-A (10.10.2.10) to App-B (10.20.1.10) and from App-B back to App-A. A controlled-failure test was performed by temporarily removing the inbound All ICMP – IPv4 rule from App-B's security group. The ping from App-A then showed 100% packet loss. After restoring the rule, the ping succeeded again with 0% packet loss. This demonstrates that the peering routes were in place and that the security group rule controlled the tested ICMP traffic.

6. Conclusion

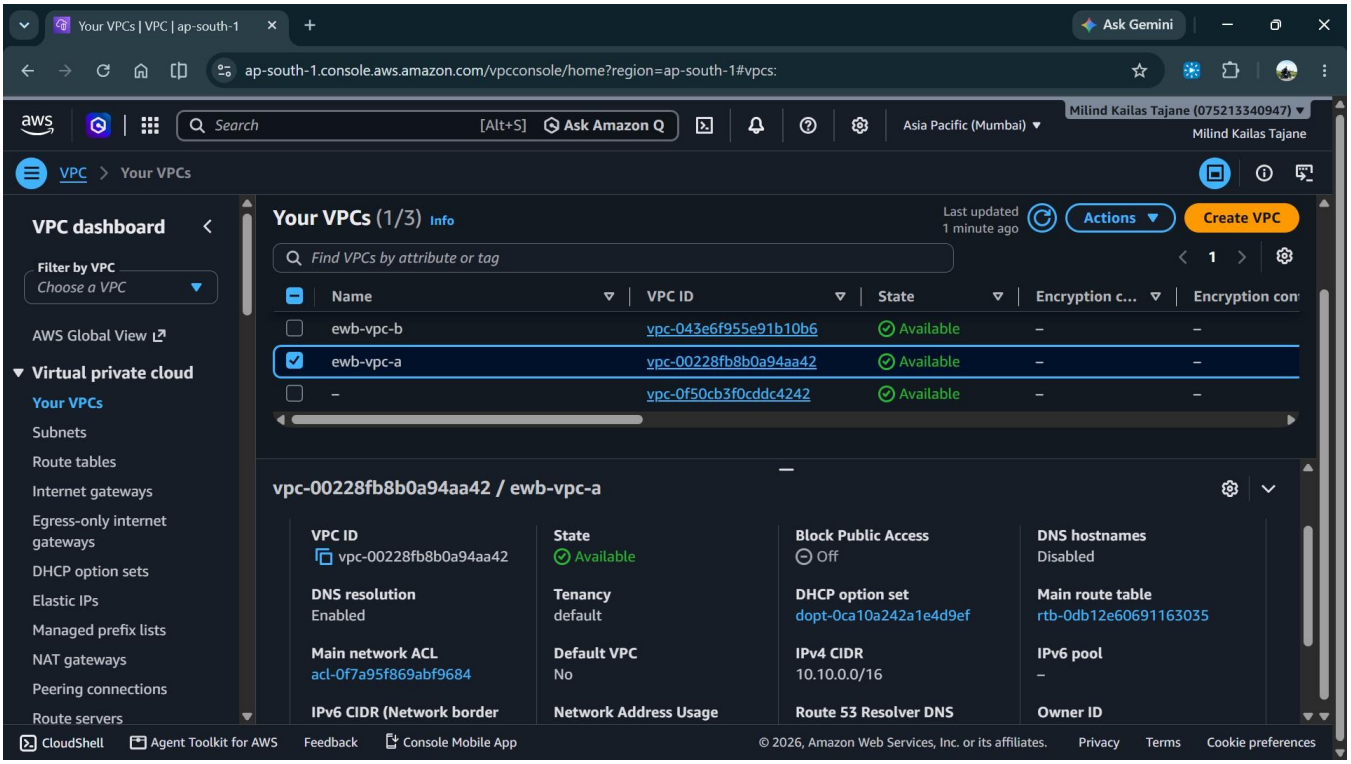
The project successfully demonstrated VPC Peering between two VPCs, routing between their private CIDR ranges, and security-group-based traffic control. Bidirectional ping tests succeeded, failed when the relevant ICMP rule was removed, and succeeded again after the rule was restored.

Public-sharing note: This version redacts the terminal lines showing the local SSH key path and fingerprint from the App-A to App-B evidence screenshot. Never publish private keys, AWS credentials, or session tokens.

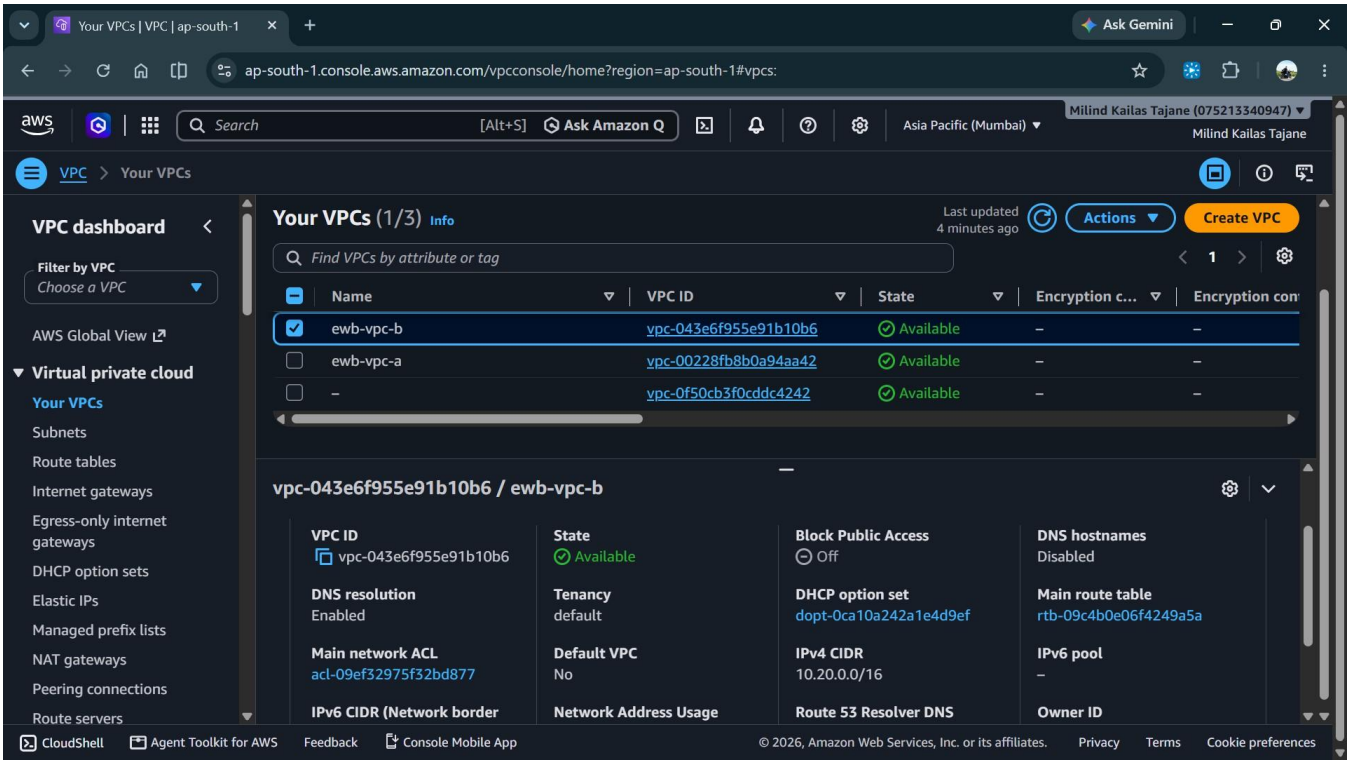
Appendix A — Evidence screenshots

The following pages contain the supplied screenshots. Images are scaled to fit the report page while preserving their aspect ratio.

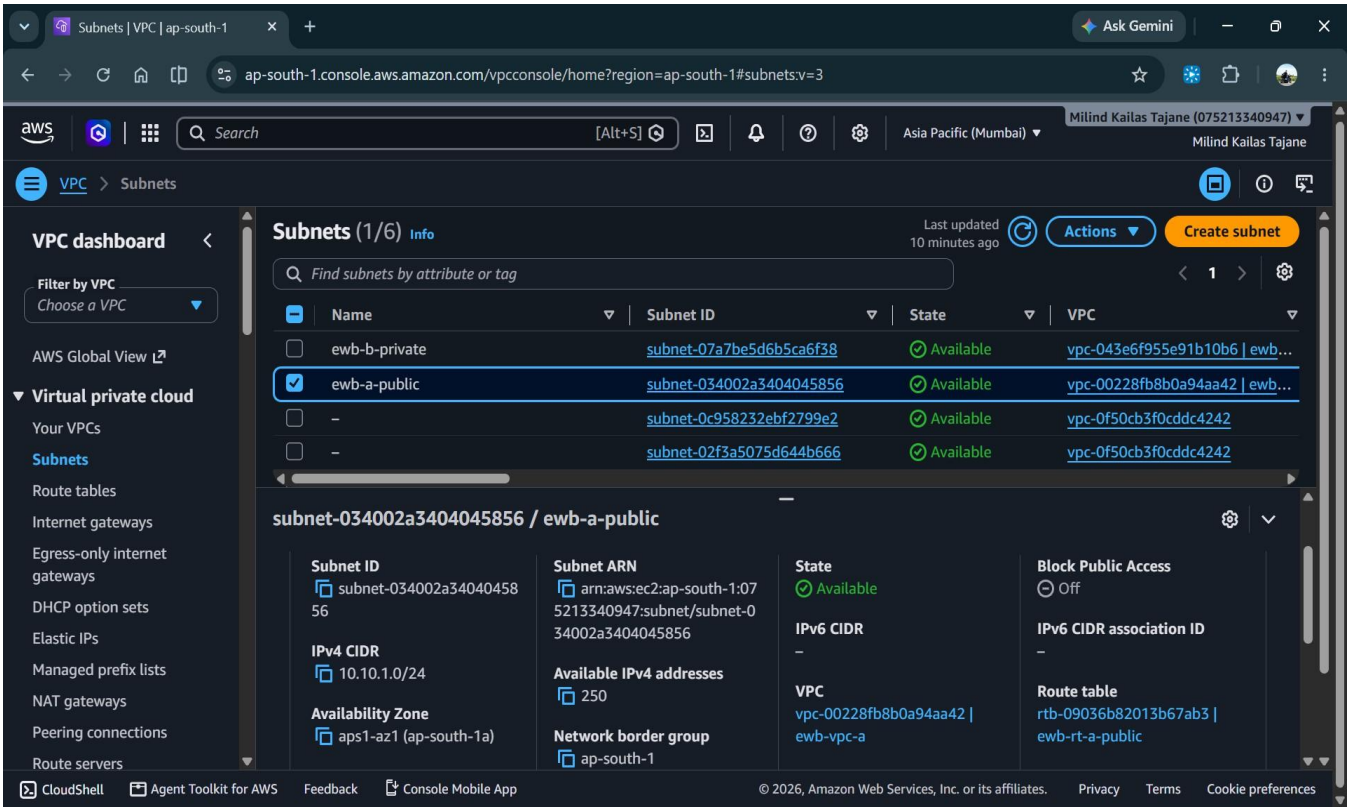
Evidence 1 — VPC A configuration



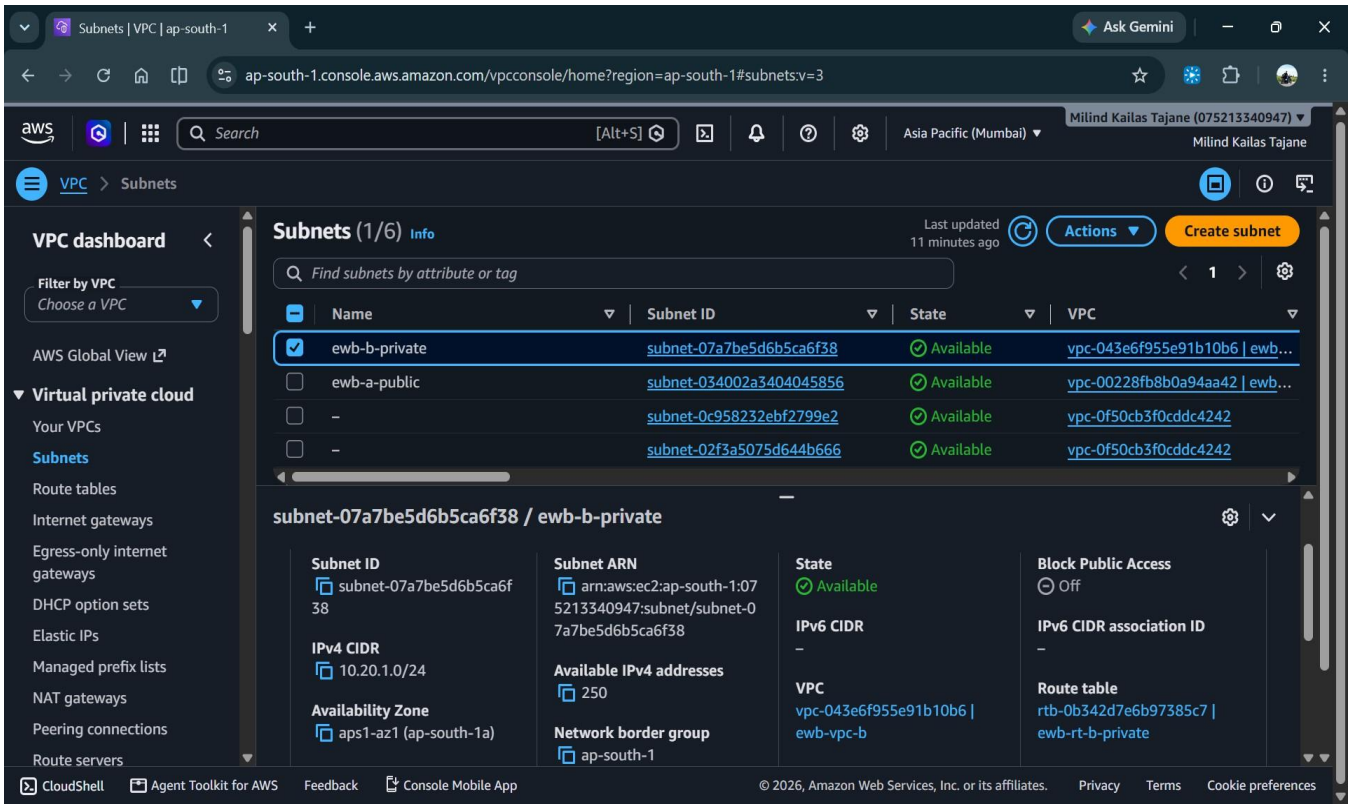
Evidence 2 — VPC B configuration



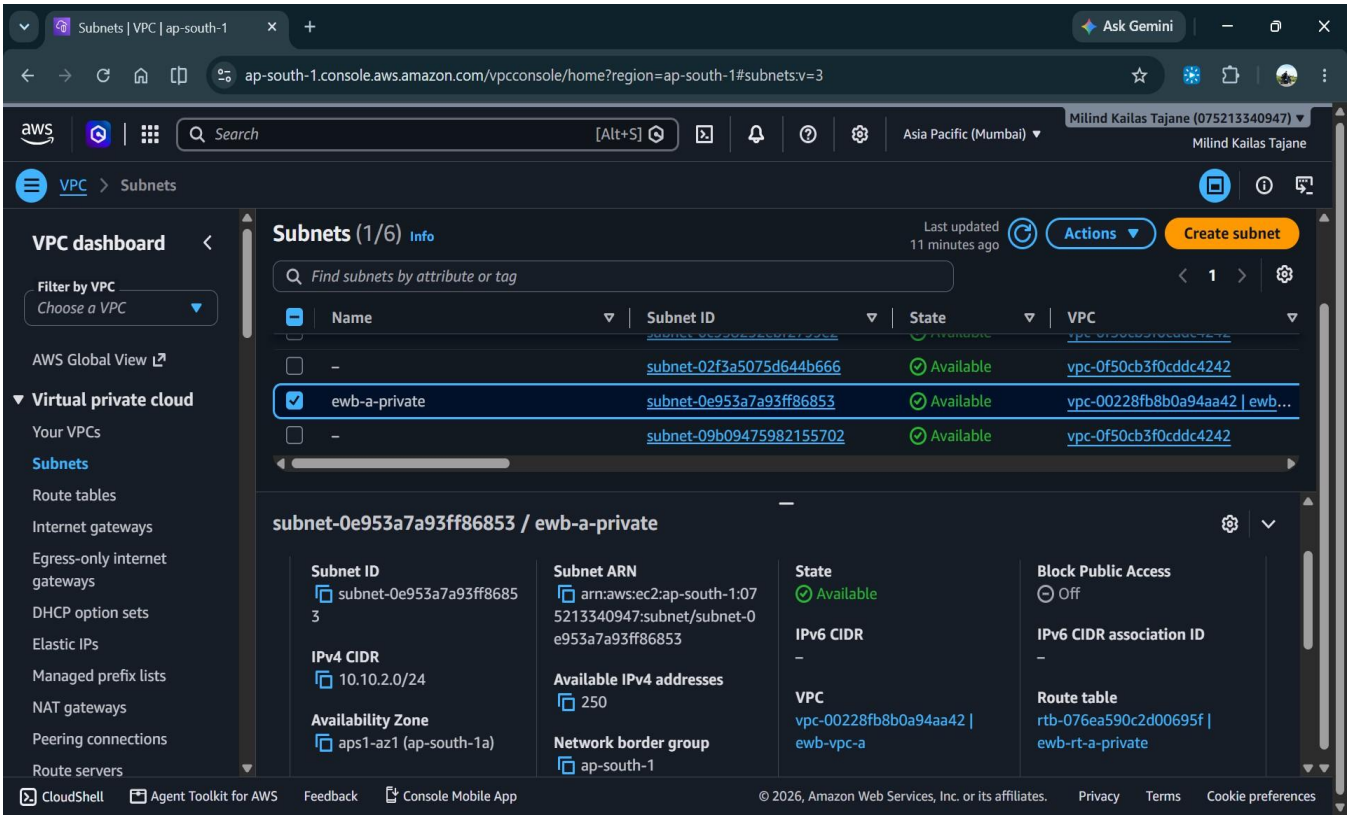
Evidence 3a — VPC A public subnet



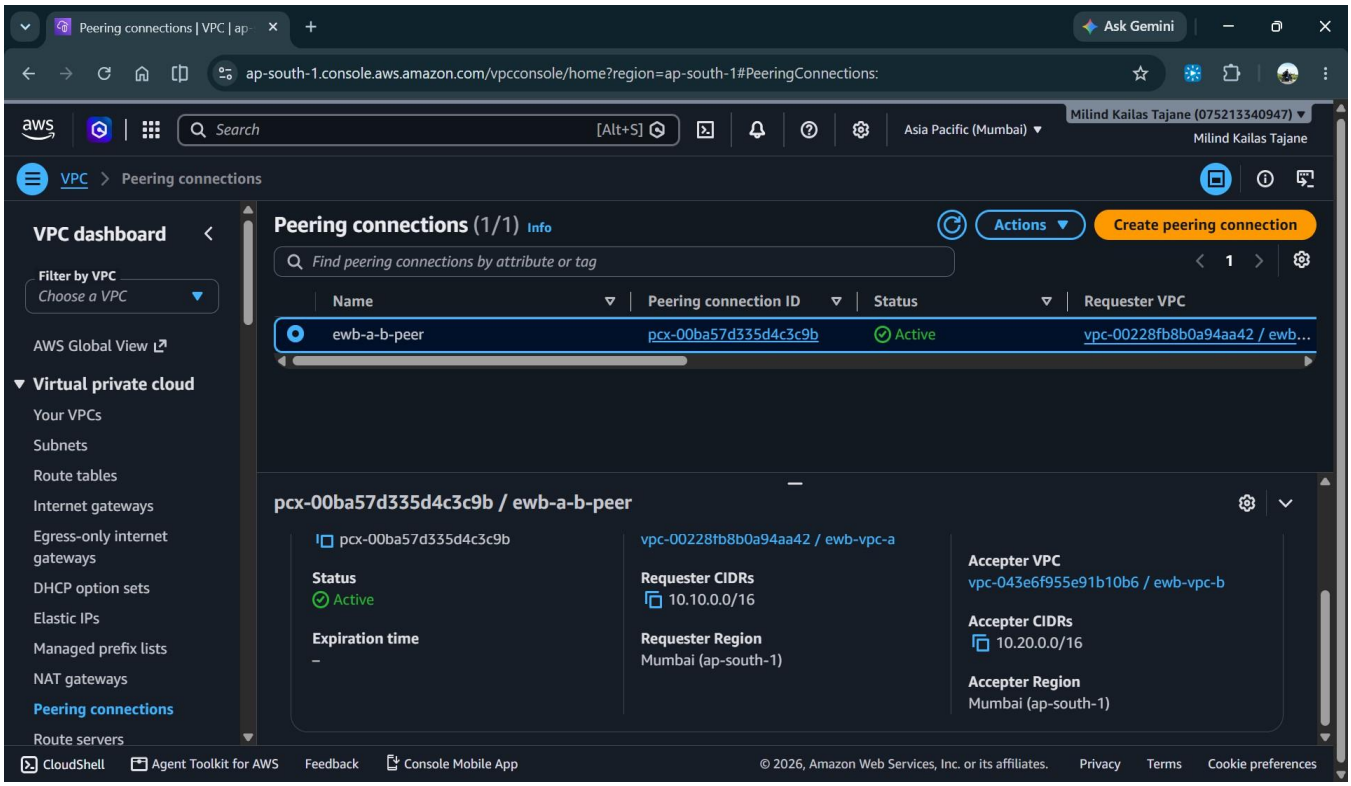
Evidence 3b — VPC B private subnet



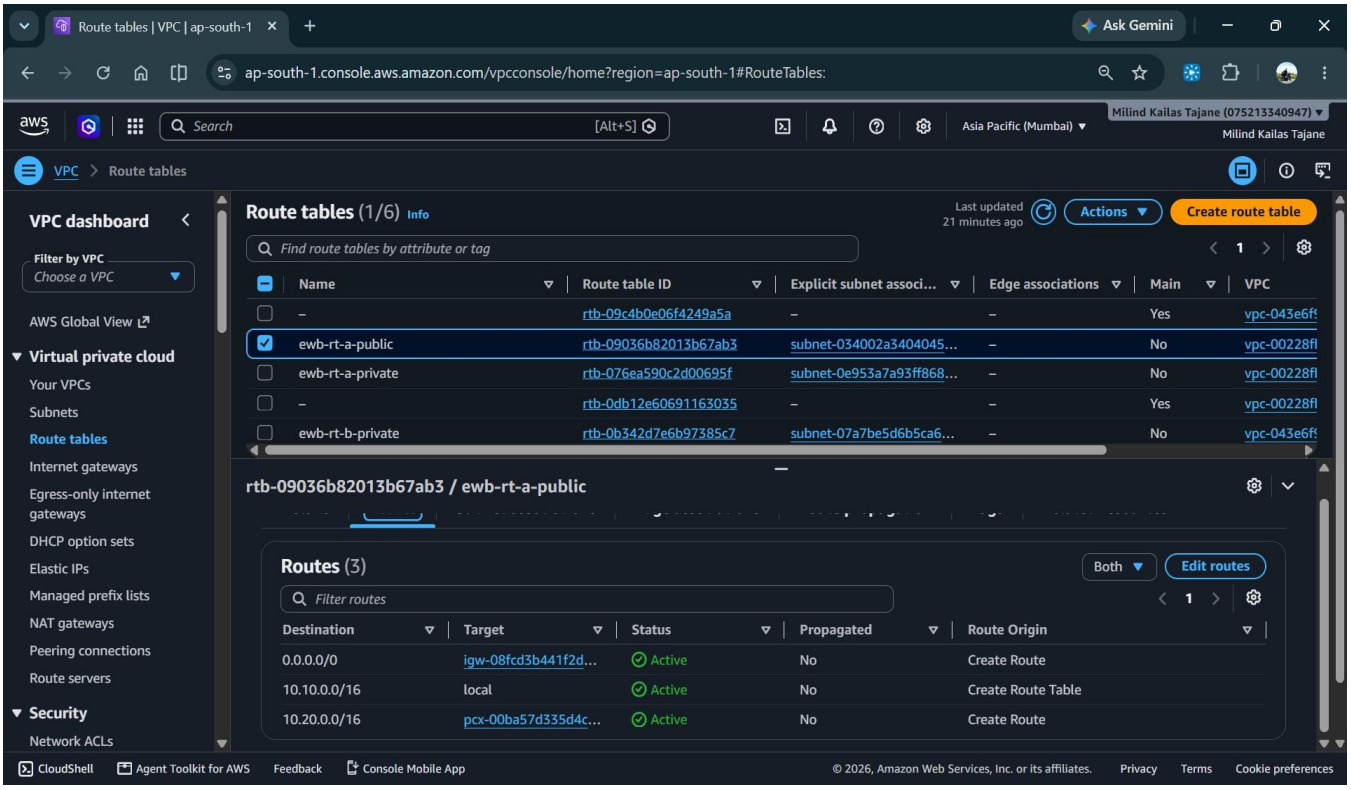
Evidence 3c — VPC A private subnet



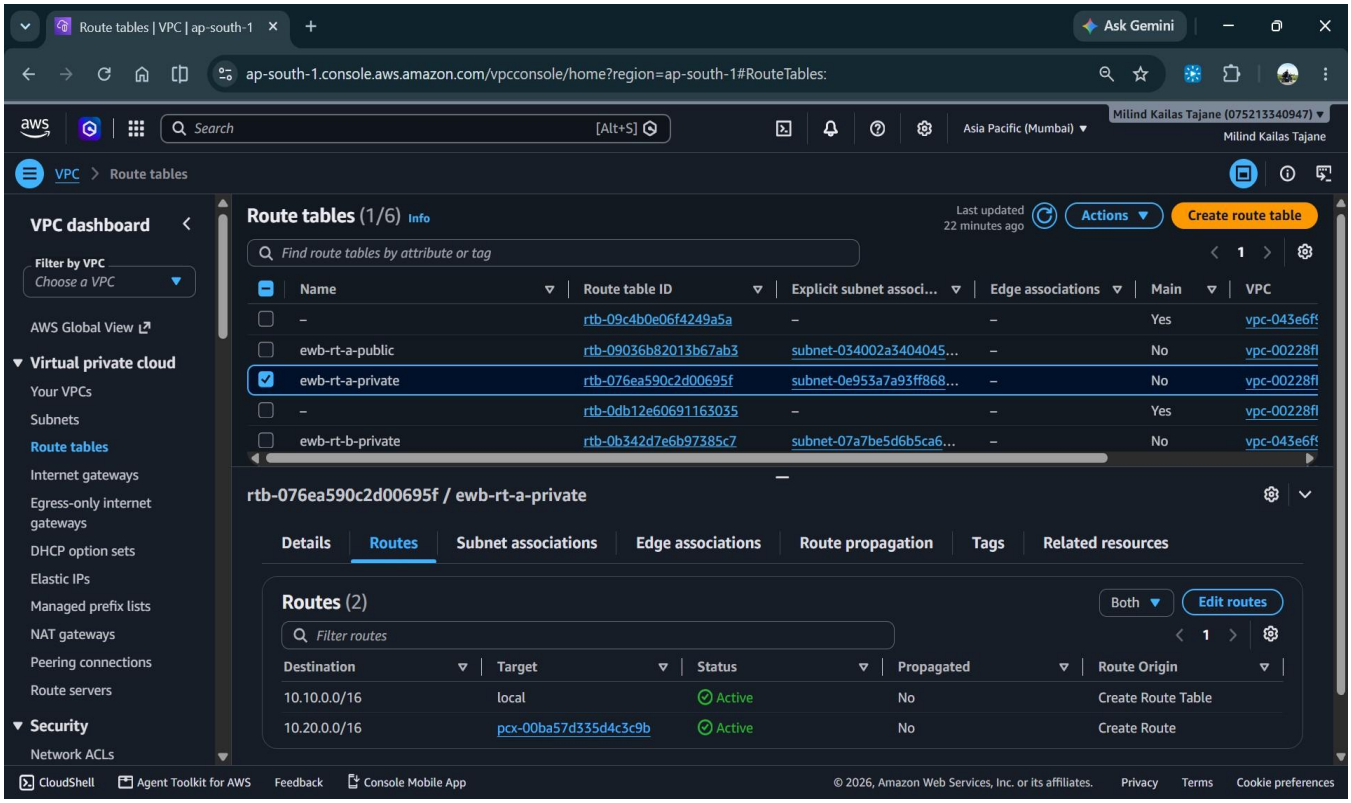
Evidence 4 — Active VPC Peering connection



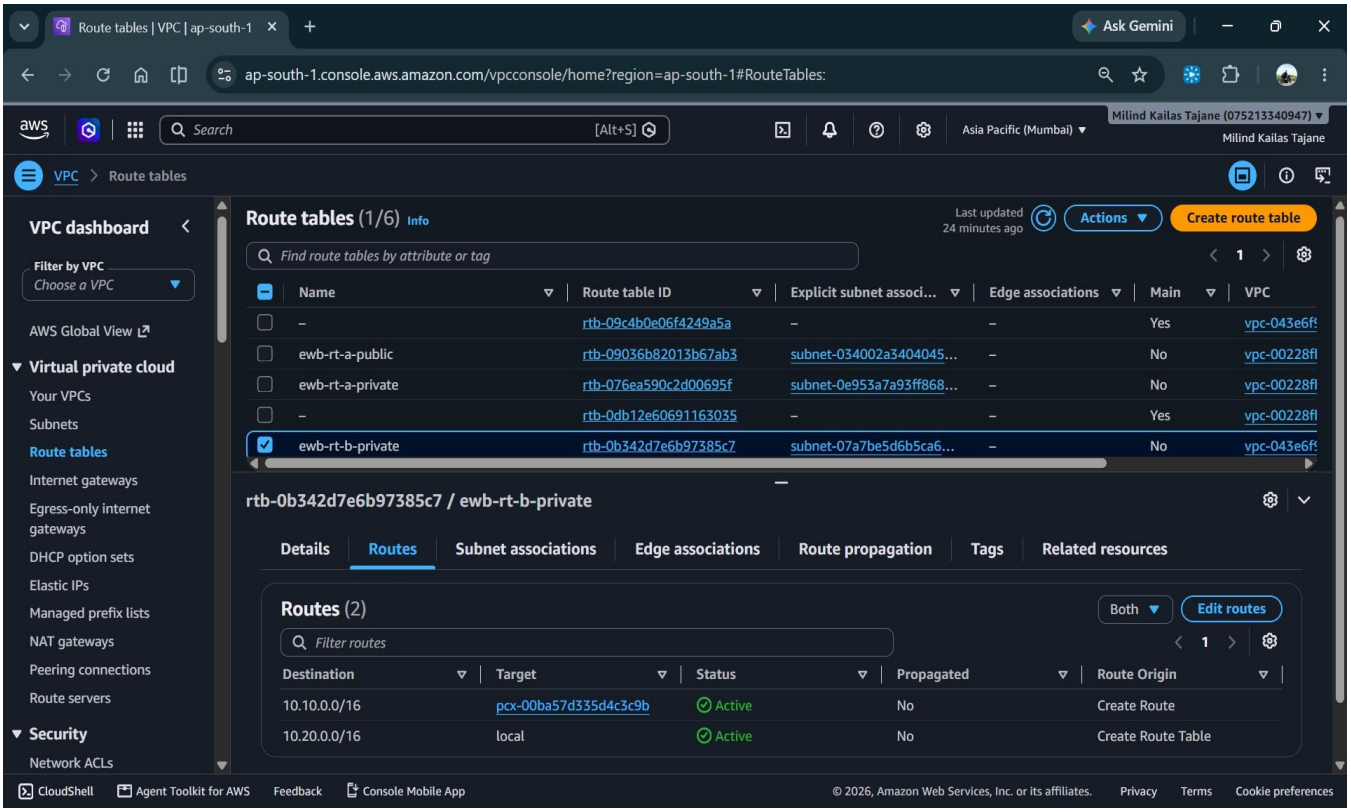
Evidence 5 — VPC A public route table



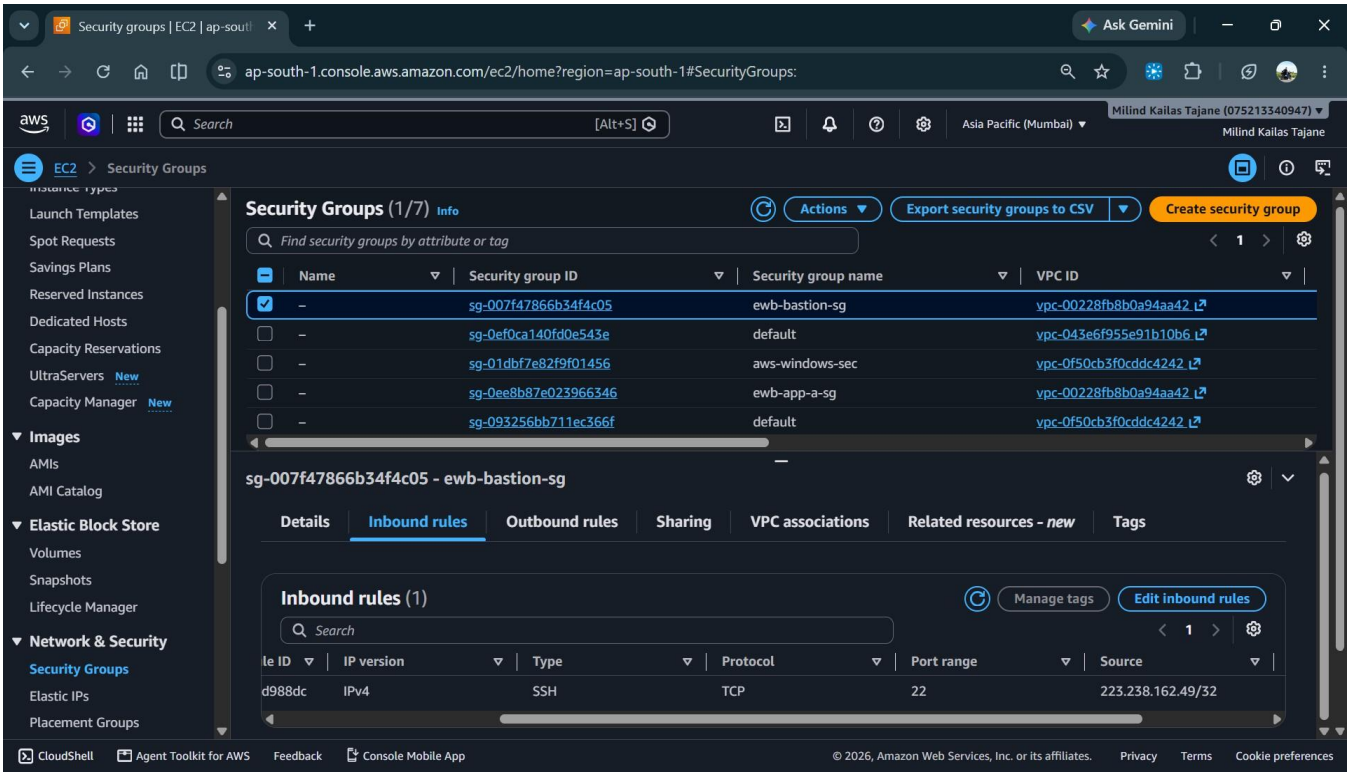
Evidence 6 — VPC A private route table



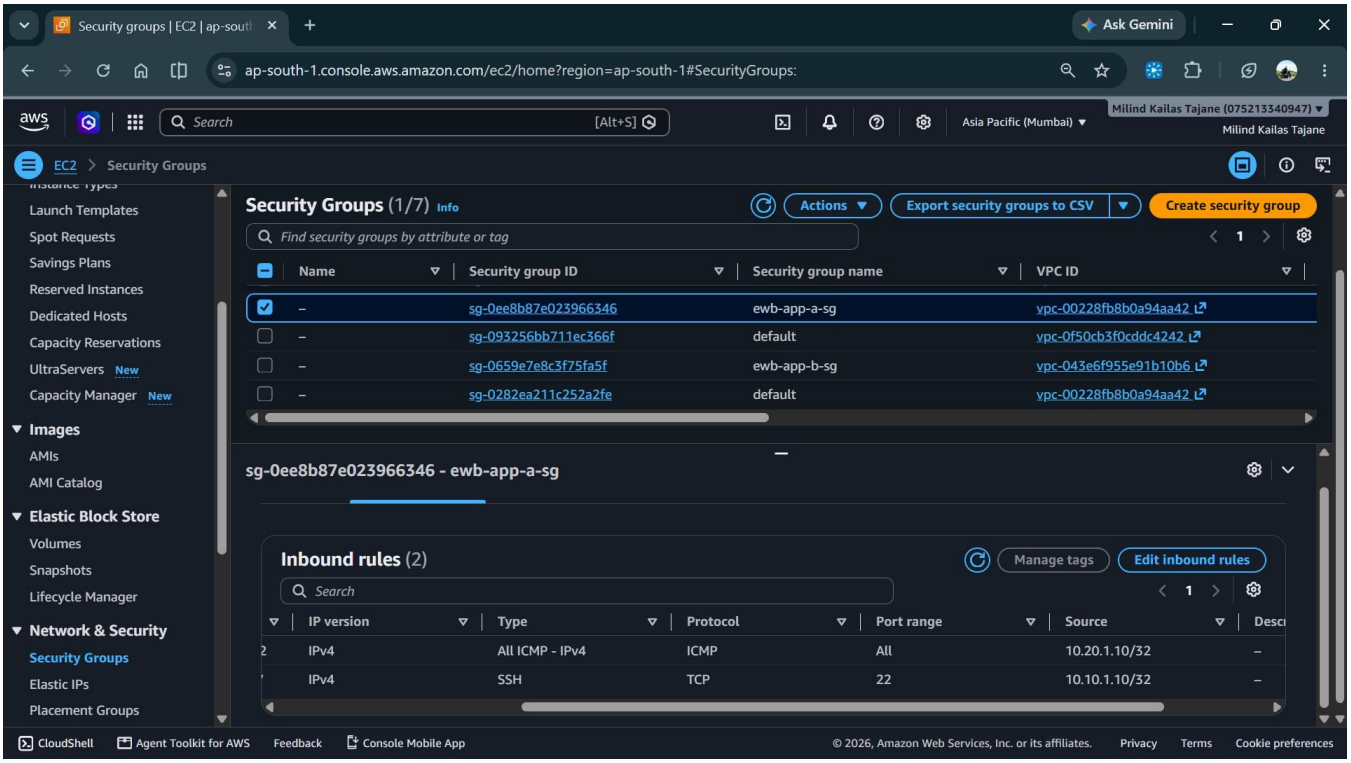
Evidence 7 — VPC B private route table



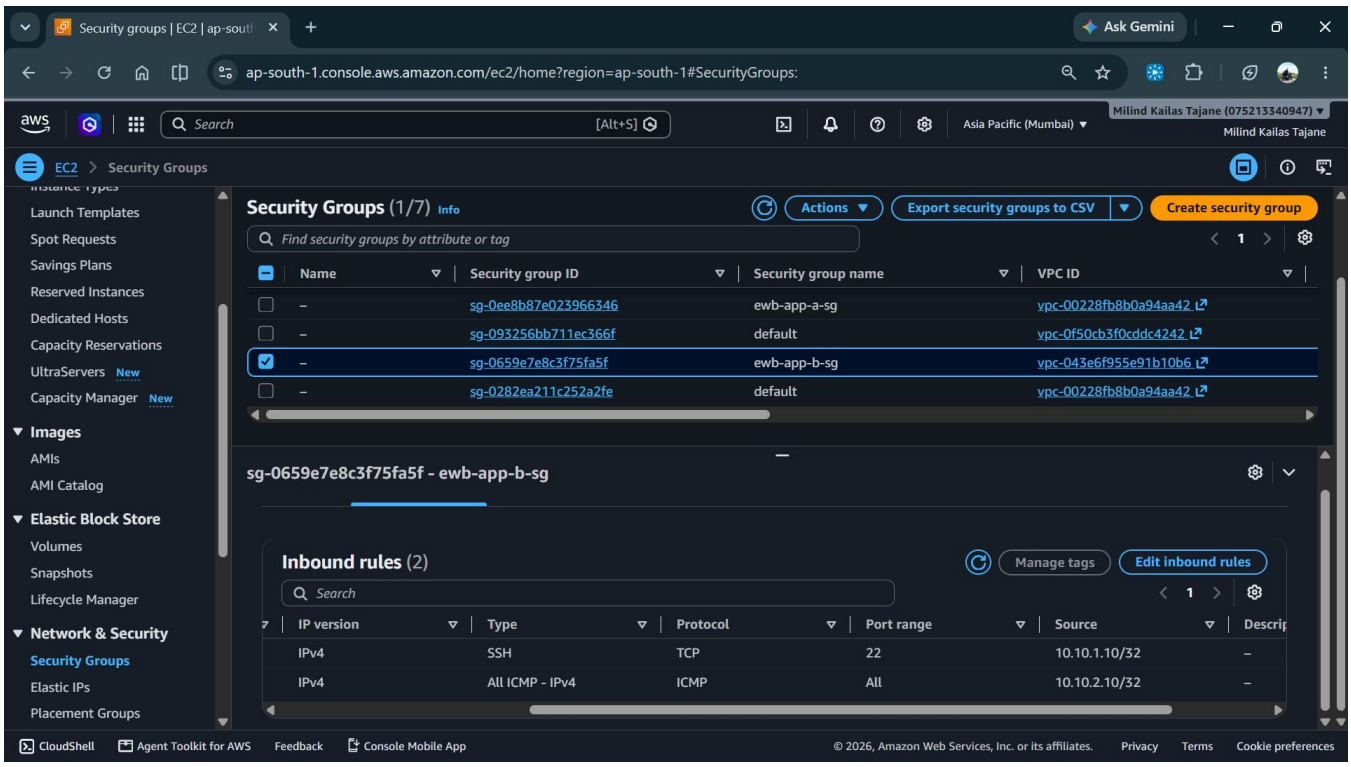
Evidence 8a — Bastion security group



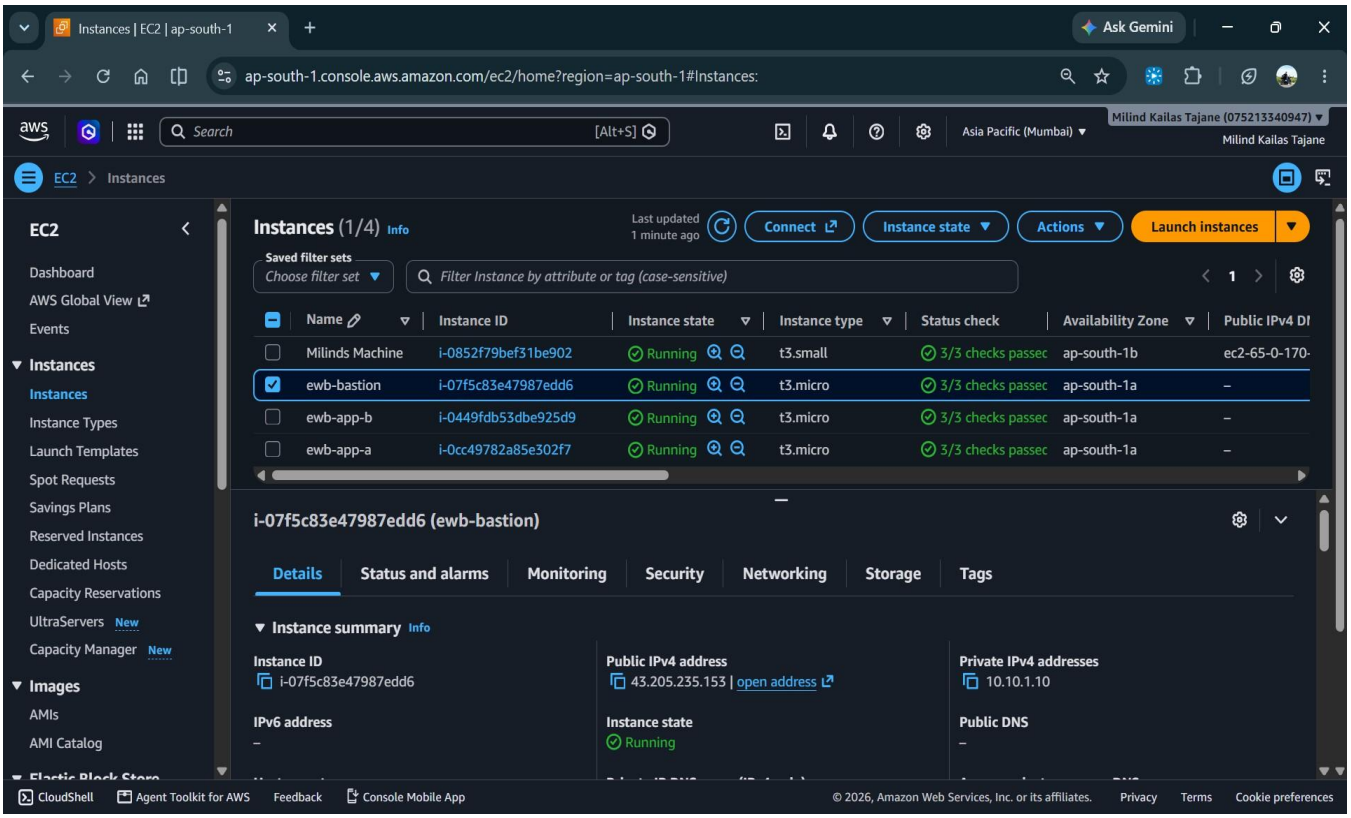
Evidence 8b — App-A security group



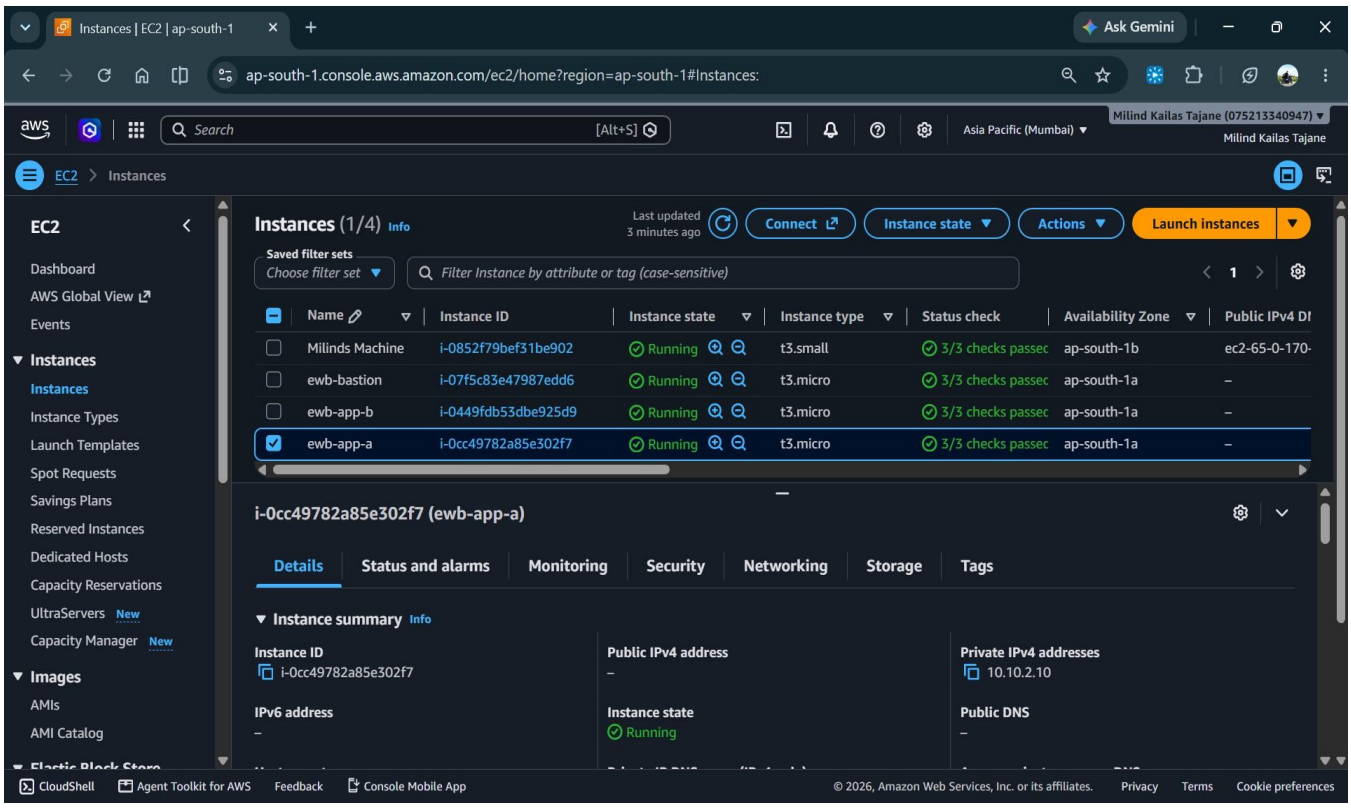
Evidence 8c — App-B security group



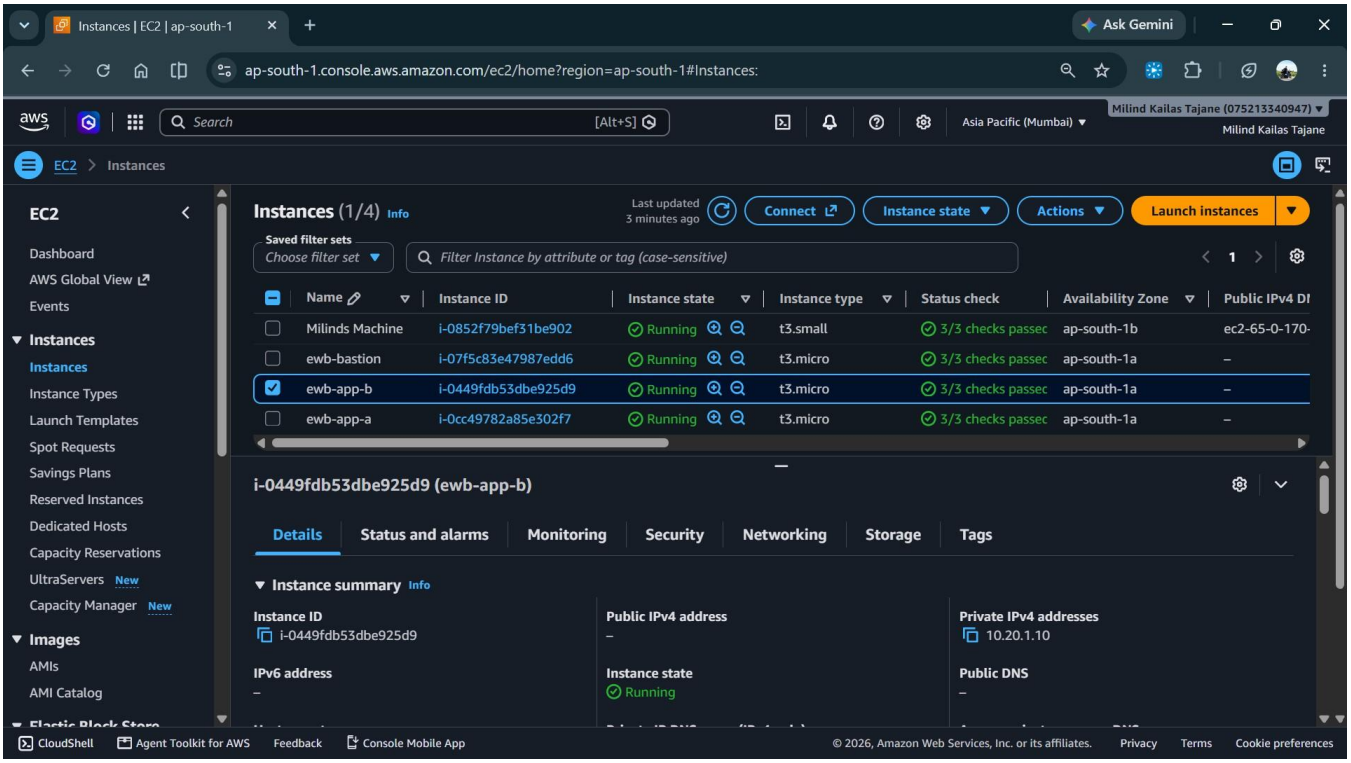
Evidence 9a — Bastion EC2 instance



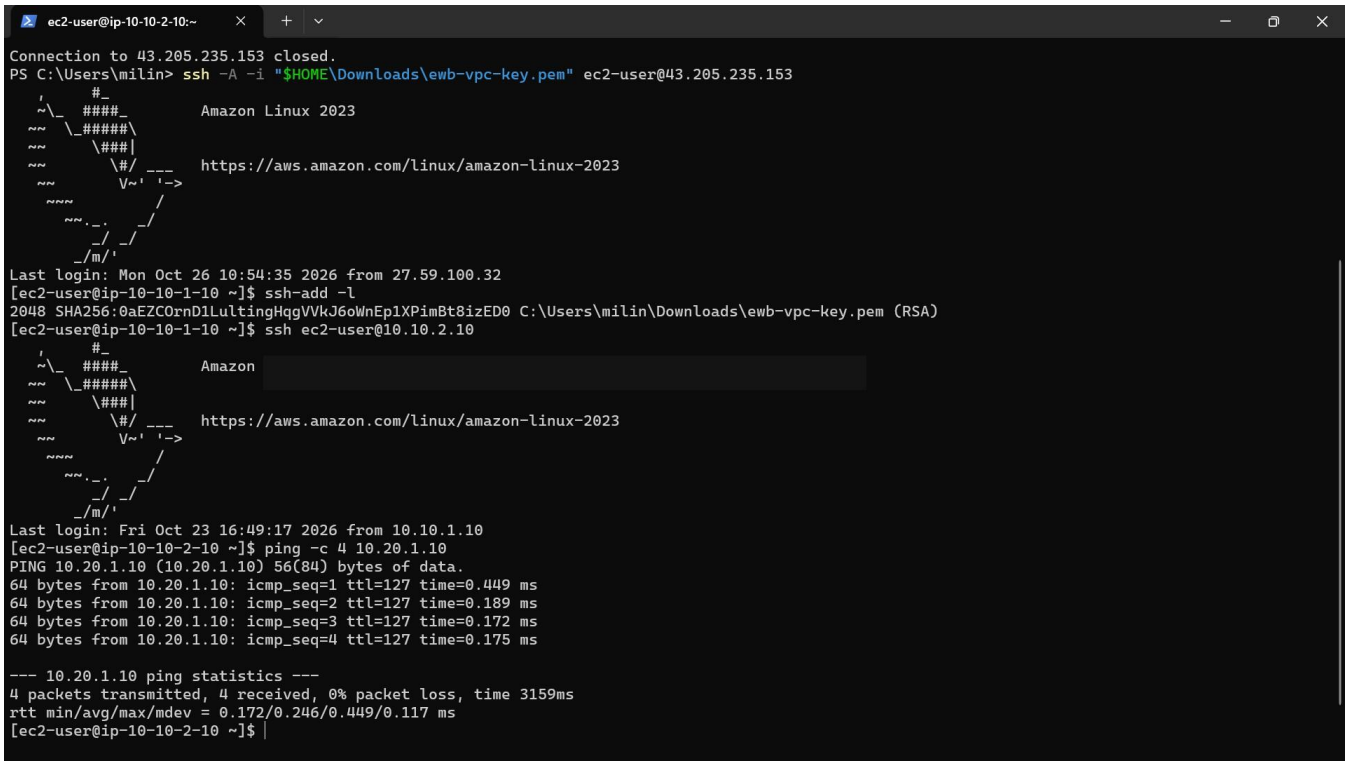
Evidence 9b — App-A EC2 instance



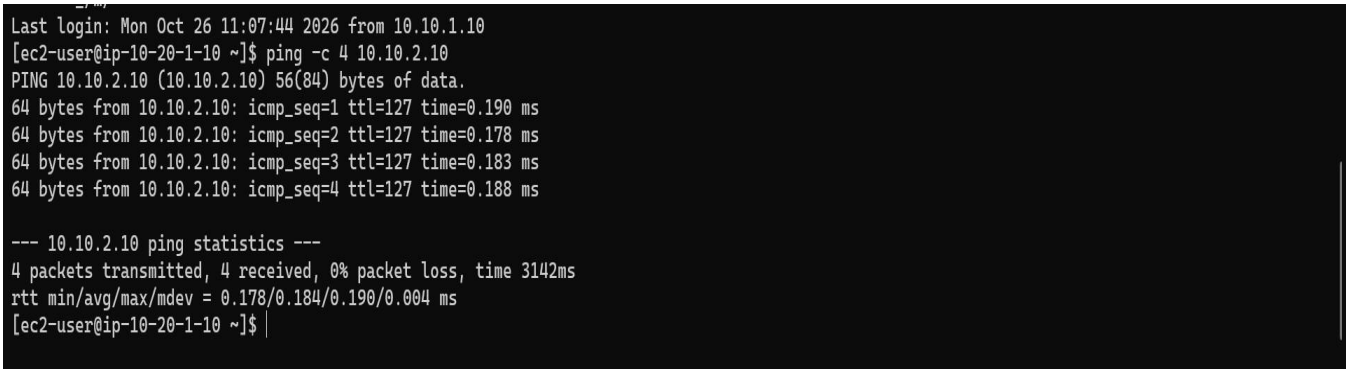
Evidence 9c — App-B EC2 instance



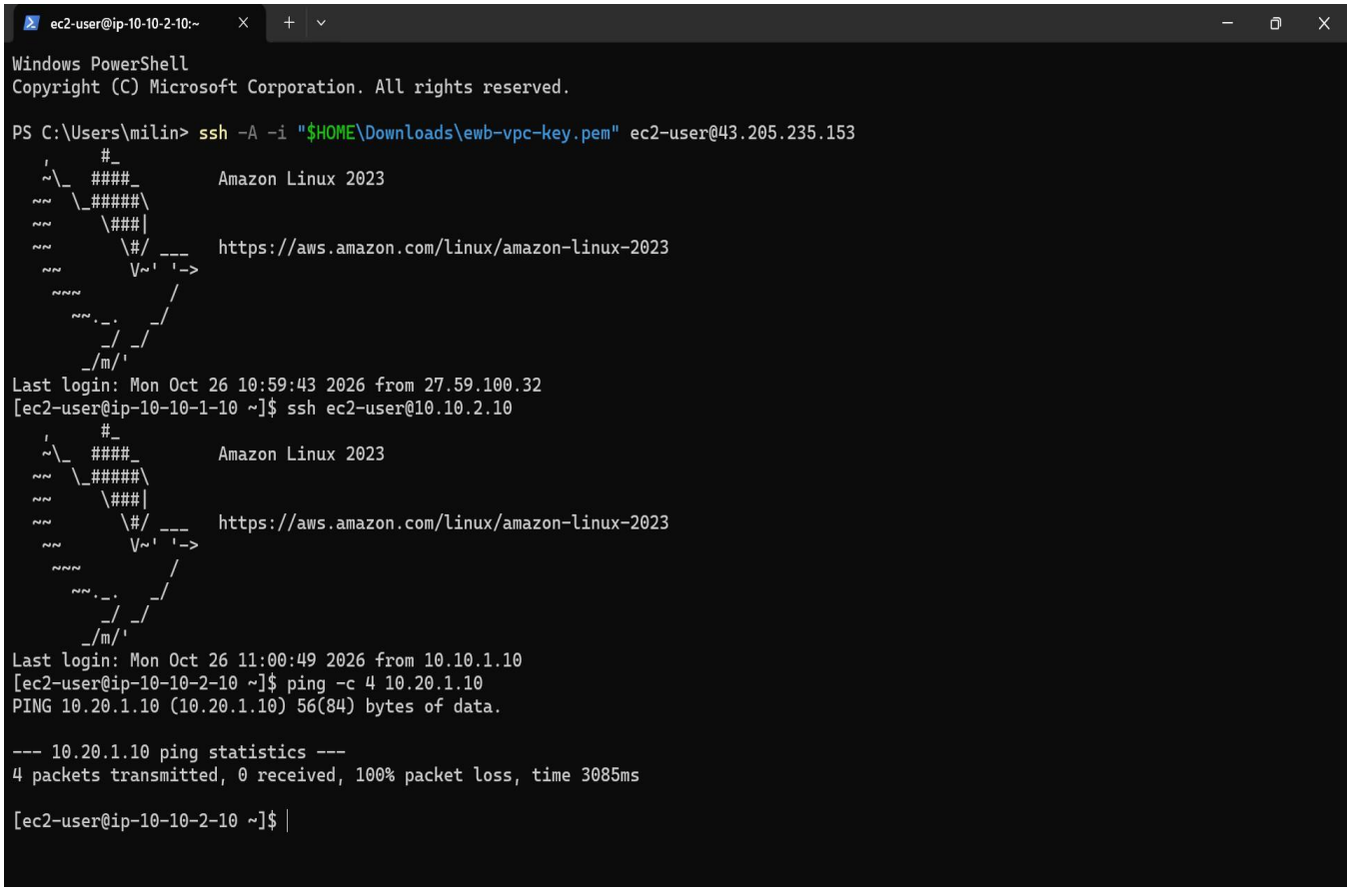
Evidence 10 — Successful ping: App-A to App-B



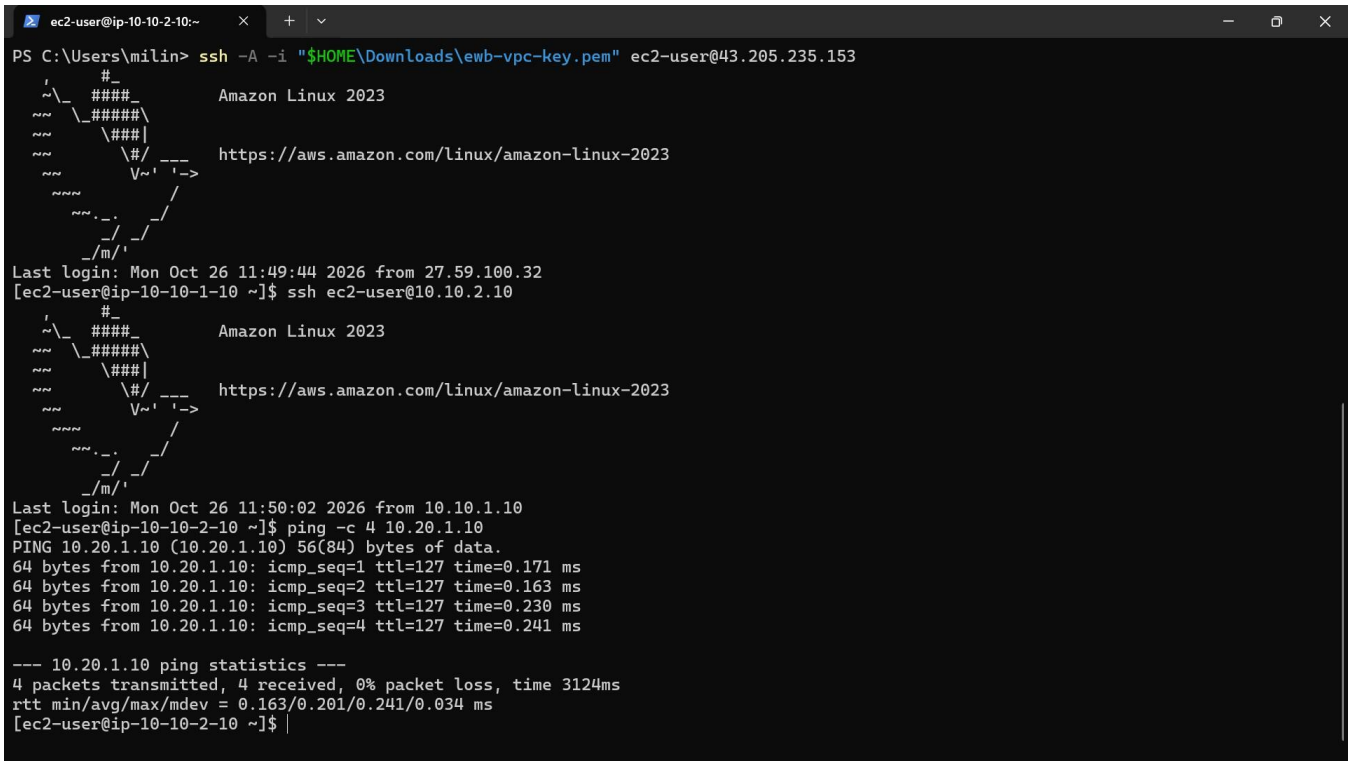
Evidence 11 — Successful ping: App-B to App-A



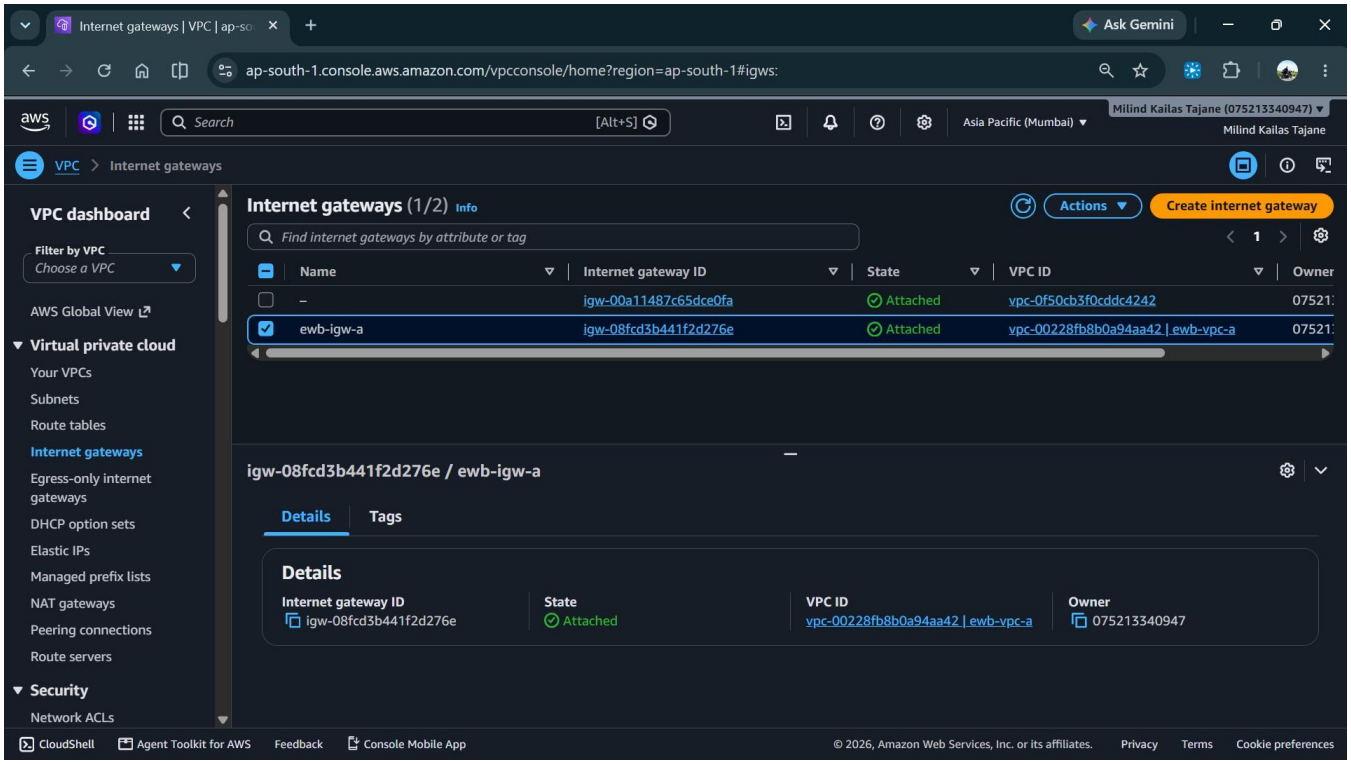
Evidence 12 — Controlled ICMP failure



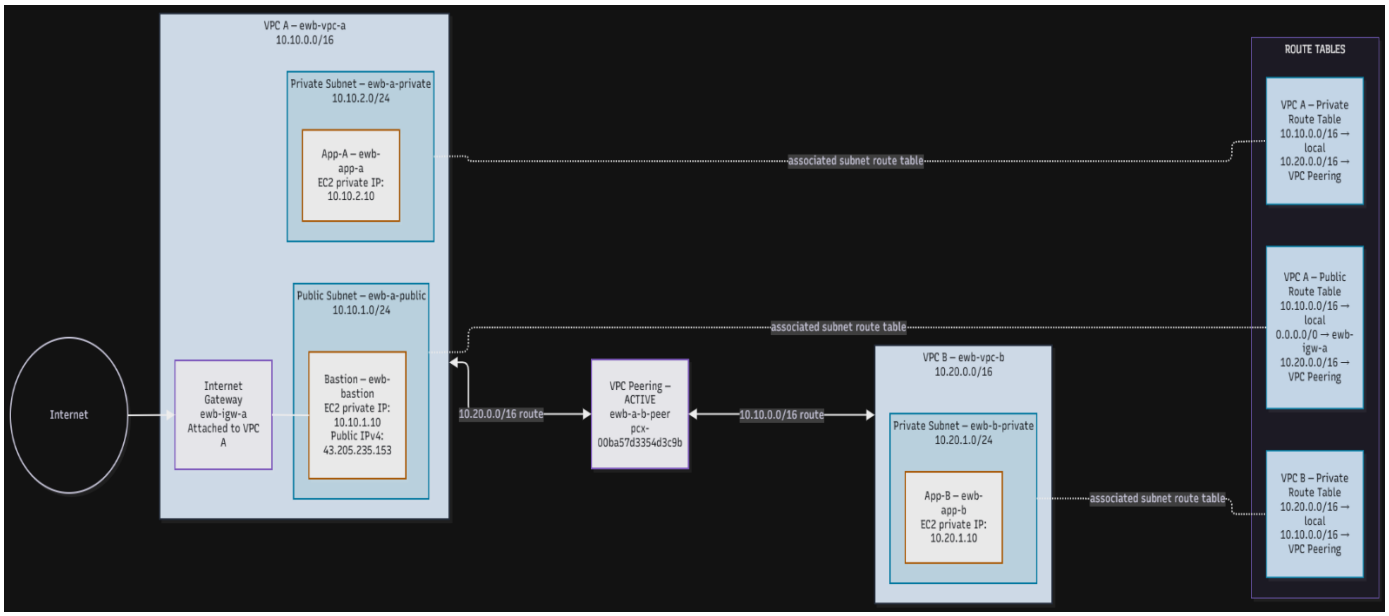
Evidence 13 — Connectivity restored



Evidence 14 — Internet Gateway attached to VPC A



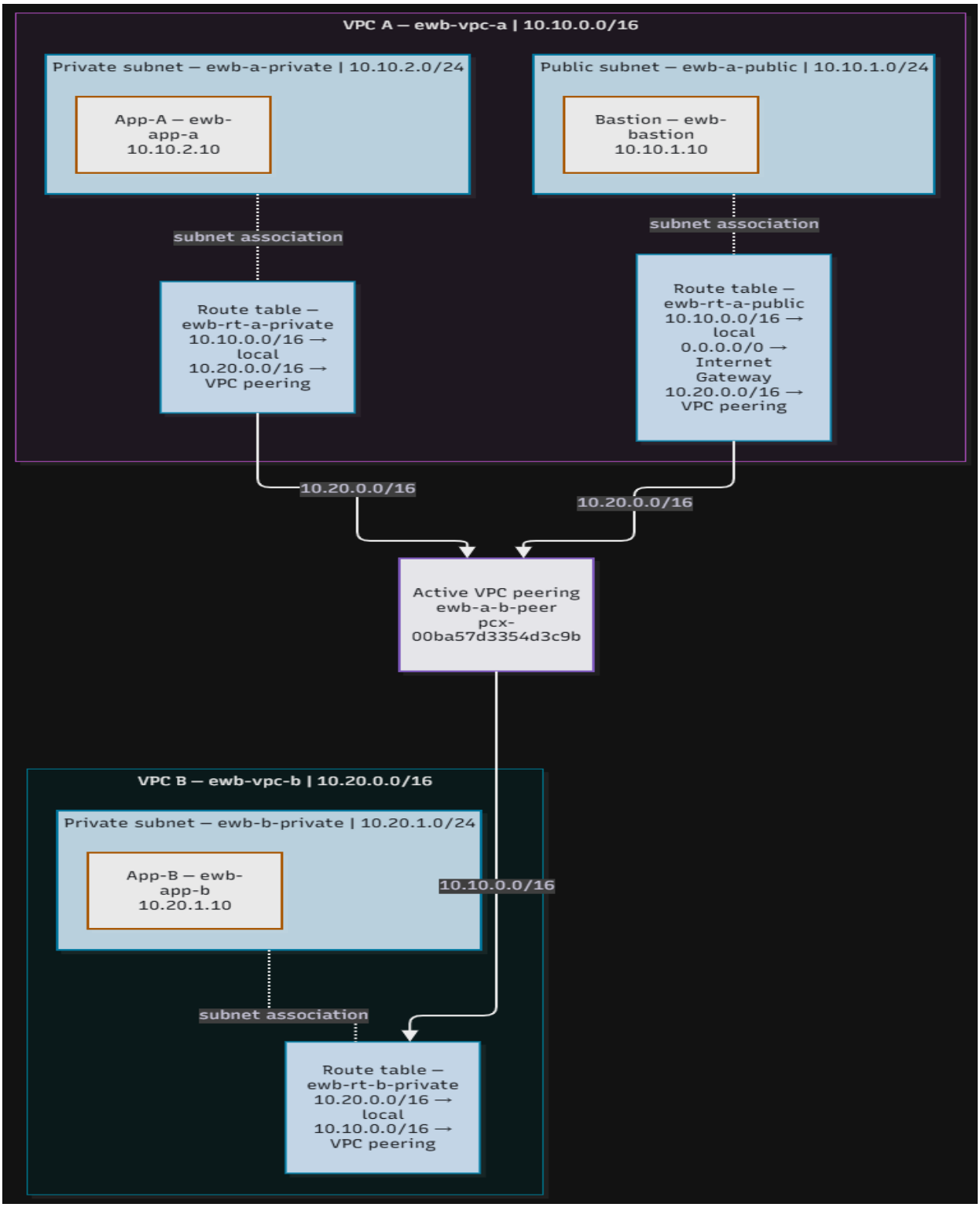
AWS VPC Peering Architecture



Architecture Overview

The architecture demonstrates communication between VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) through an active VPC peering connection. Route tables direct traffic between the two networks, while security groups control permitted traffic. App-A (10.10.2.10) and App-B (10.20.1.10) were used to verify connectivity in both directions.

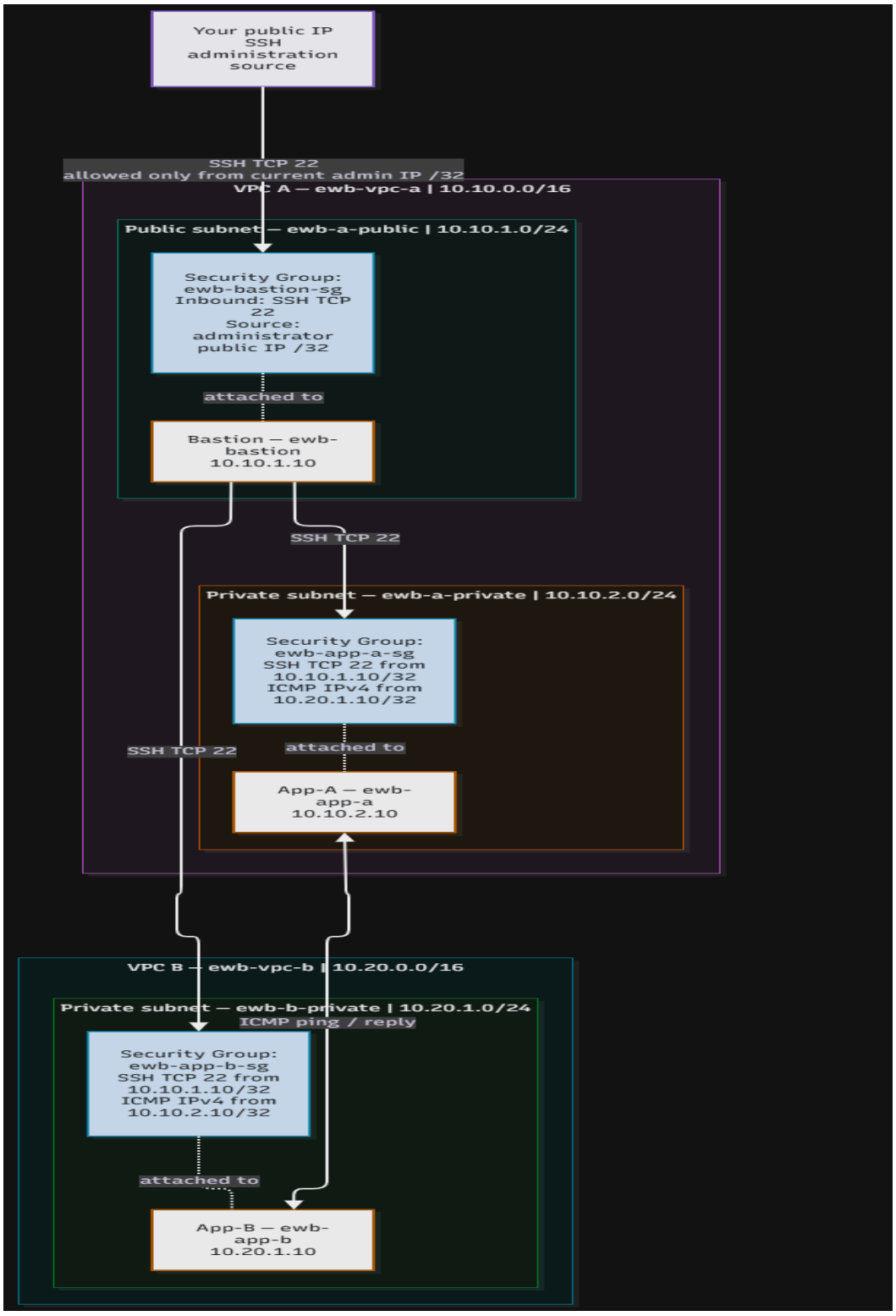
VPC Route Tables and Traffic Routing



Routing Overview

The route tables direct traffic between VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) through the active VPC peering connection. The VPC A public route table also includes a default route to the Internet Gateway. The private route tables contain routes to the remote VPC CIDR through the peering connection.

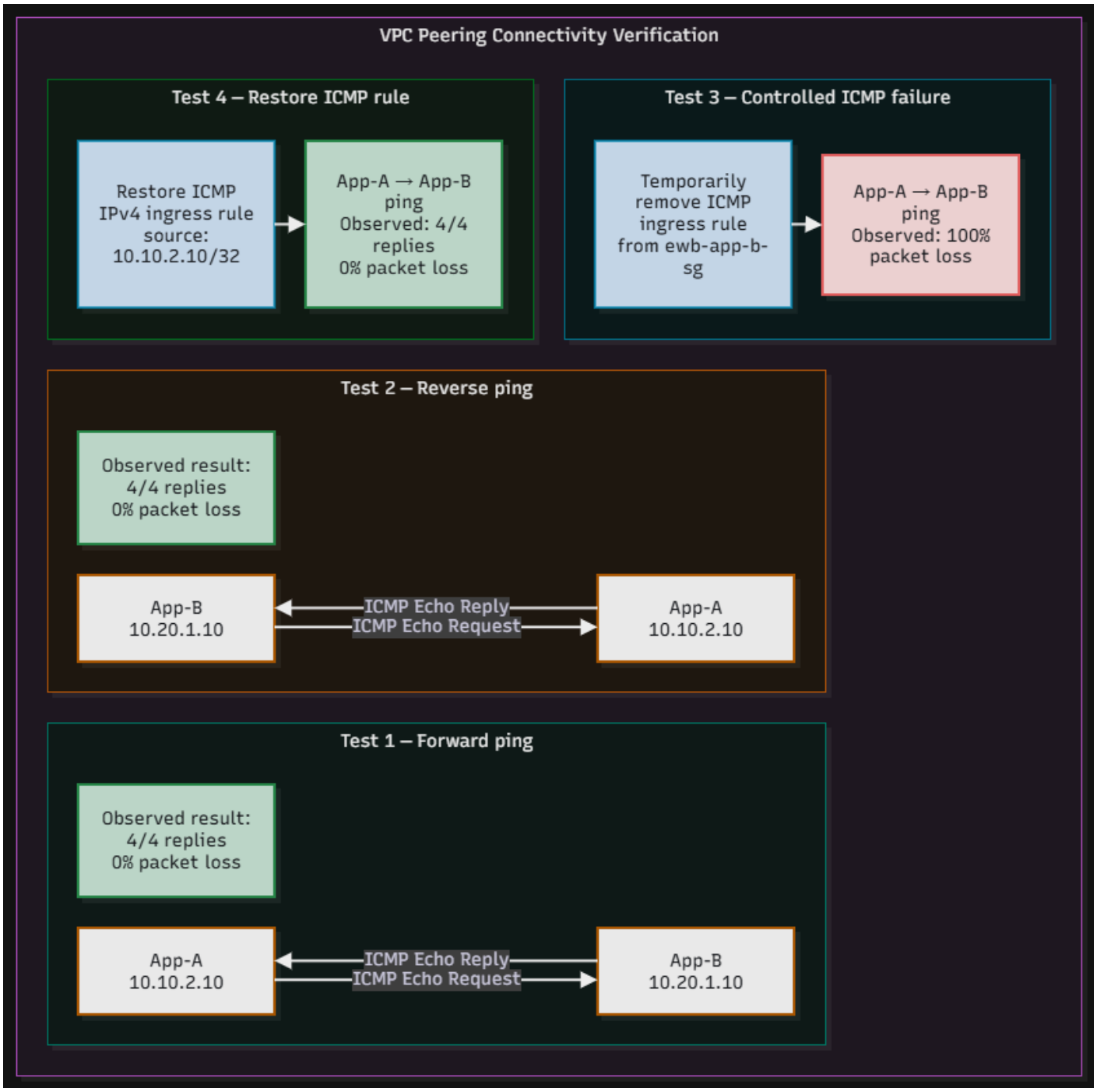
Security Groups and Access Control



Security Controls Overview

Security groups control access to the EC2 instances used in this lab. SSH access is restricted to the configured source, while ICMP rules permit the intended connectivity tests between App-A and App-B. The controlled-failure test demonstrated that removing the relevant ICMP rule interrupted ping traffic, and restoring the rule allowed connectivity to succeed again.

Connectivity Testing, Failure and Recovery



Testing and Recovery Summary

Connectivity was tested between App-A (10.10.2.10) and App-B (10.20.1.10) in both directions. The ping tests succeeded when the required ICMP security group rule was present. Temporarily removing the relevant ICMP rule caused the test to fail, and restoring the rule returned connectivity to a successful state.

AWS Resource Cleanup and Cost Management

Objective

The objective of cleanup is to remove temporary AWS resources created for the EWB VPC Peering Student Project and prevent unnecessary charges after project completion.

Cleanup Checklist

The following resources must be reviewed and cleaned up after project evidence has been saved and the mentor confirms that the lab environment is no longer required:

1. Terminate the project EC2 instances: ewb-bastion, ewb-app-a, and ewb-app-b.
2. Check for and delete unused EBS volumes associated with the project.
3. Release any Elastic IP addresses created for the project, if applicable.
4. Remove the VPC peering connection after saving the required evidence.
5. Remove project-specific routes, security groups, and route tables.
6. Detach and delete the Internet Gateway created for the project.
7. Delete the project subnets and VPCs after confirming that no required resources remain.
8. Verify that the project resources have been removed from the AWS account.

Cleanup Status

The cleanup status must be verified in the AWS Management Console before it can be marked as completed. Until verification is performed, the cleanup is considered pending.

Cost Management and Safety

Only resources created specifically for this project should be removed. Unrelated AWS resources must be preserved. In particular, the unrelated instance named Milinds Machine must not be modified or terminated.

Conclusion

The cleanup process is an important final step in responsible AWS resource management. Verifying the removal of temporary resources helps prevent unnecessary costs and ensures that the AWS environment is left in the intended state.

Learning Summary

Through this project, I learned how to create and configure two Amazon VPCs with non-overlapping CIDR ranges, establish VPC peering, configure route tables and security groups, and validate private communication using bidirectional ICMP tests. I also practiced accessing private instances through a bastion host using SSH agent forwarding and troubleshooting connectivity by removing and restoring an ICMP rule. This project helped me understand that VPC peering alone does not guarantee connectivity; the appropriate routes and security rules must also be configured. I also learned the importance of cleaning up unused AWS resources to avoid unnecessary charges.